

SISTEMA WEB PARA GERENCIAMENTO DE ESTACIONAMENTOS COM CONTROLE DE ENTRADA, SAÍDA E OCUPAÇÃO DE VAGAS

Andres Gonçalves Vitt <andresgvitt@rede.ulbra.br>

Daniel Souza Vargas <daniel.rsouza@ulbra.br> – Orientador

Universidade Luterana do Brasil (Ulbra) – Curso Análise e Desenvolvimento de Sistemas – Campus Torres

Rua Universitária, 1900 · Parque do Balonismo · CEP 95.560-000 · Torres/RS

04 de agosto de 2026

1 INTRODUÇÃO

Os municípios do Litoral Norte do Rio Grande do Sul apresentam uma frota significativa de automóveis, contribuindo para a demanda por serviços de estacionamento. Segundo o Departamento Estadual de Trânsito do Rio Grande do Sul (DETRAN/RS), em fevereiro de 2024, Torres possuía 14.516 automóveis registrados, enquanto Capão da Canoa contabilizava 22.044 e Arroio do Sal 3.520. Além da frota local, esses municípios recebem um grande fluxo de visitantes durante a temporada de verão, aumentando significativamente a utilização de estacionamentos e intensificando a necessidade de um controle eficiente das operações. Apesar dessa demanda, muitos estacionamentos de pequeno e médio porte ainda não utilizam sistemas informatizados de gestão, seja pelo custo de implantação ou pela necessidade de infraestrutura específica, como cancelas automáticas e equipamentos dedicados.

Em muitos estacionamentos de pequeno e médio porte, o controle das operações ainda é realizado por meio de anotações em papel ou planilhas eletrônicas. Quando um veículo chega, o funcionário precisa verificar manualmente se há vagas disponíveis e registrar a entrada e o horário de entrada. No momento da saída, o tempo de permanência é calculado manualmente, aumentando a possibilidade de erros no valor cobrado ao cliente. Além disso, a ausência de um registro estruturado das movimentações dificulta a conferência do caixa e a identificação de divergências financeiras ao final do expediente, uma vez que nem sempre é possível localizar em qual atendimento ocorreu o erro. Outro problema ocorre quando há alteração nas tarifas praticadas pelo estacionamento: sem o registro do valor aplicado em cada atendimento, torna-se difícil reconstituir corretamente o faturamento de períodos anteriores e auditar os valores recebidos.

Diante desse cenário, este Projeto Tecnológico propõe o desenvolvimento de um sistema web para gerenciamento de estacionamentos de pequeno e médio porte. O sistema terá como finalidade registrar a entrada e a saída de veículos, controlar a ocupação das vagas, calcular automaticamente o valor da permanência conforme a tarifa vigente no momento do

atendimento e manter o histórico das operações. O sistema contemplará ainda o controle financeiro da operação, por meio do fechamento de caixa e da geração de relatórios, e o envio de notificações automáticas ao condutor quando a permanência ultrapassar o período configurado.

Para o desenvolvimento da solução, será utilizada uma aplicação web composta por um frontend desenvolvido em React, permitindo a atualização dinâmica das informações de vagas e movimentações sem a necessidade de recarregar a página. O backend será implementado em Node.js com Express, responsável pelo processamento das regras de negócio e pela comunicação com o banco de dados PostgreSQL, escolhido por oferecer suporte a transações, característica importante para o controle das movimentações financeiras do estacionamento.

2 FUNDAMENTAÇÃO TEÓRICA

2.1 Arquitetura cliente-servidor e APIs REST

A arquitetura cliente-servidor divide as responsabilidades do sistema entre o cliente, responsável pela interface com o usuário e pelo envio das requisições, e o servidor, responsável pelo processamento das regras de negócio, pelo acesso aos dados e pelo retorno das respostas. Essa separação favorece a organização do sistema, facilita sua manutenção e permite que diferentes clientes consumam os mesmos serviços.

A comunicação entre cliente e servidor pode ser realizada por meio de APIs que seguem o estilo arquitetural REST (Representational State Transfer). Segundo Fielding (2000), REST é um estilo arquitetural para sistemas distribuídos baseado em restrições como cliente-servidor, comunicação sem estado (stateless), sistema em camadas, cache e interface uniforme. Na Web, essas restrições são normalmente implementadas utilizando o protocolo HTTP, no qual os recursos são identificados por URIs e manipulados por métodos como GET, POST, PUT e DELETE.

No sistema proposto para o gerenciamento de estacionamentos, a arquitetura cliente-servidor é adotada por meio de uma aplicação web responsável pela interação com os operadores e de um servidor encarregado do processamento das regras de negócio e do acesso ao banco de dados. A comunicação entre esses componentes ocorre por meio de uma API REST, responsável por operações como cadastro de vagas, registro de entrada e saída de veículos, cálculo do valor da permanência e registro dos pagamentos. No backend, a aplicação é organizada em camadas, separando as rotas responsáveis pelo atendimento das requisições HTTP, a camada de serviços onde são implementadas as regras de negócio e a camada de acesso

aos dados, responsável pelas operações no banco de dados. Essa organização favorece a manutenção do código, a reutilização de componentes e a realização de testes nas regras de negócio de forma independente da interface e da persistência dos dados.

Embora arquiteturas baseadas em microsserviços ofereçam vantagens em aplicações de grande porte, sua adoção não se justifica neste projeto. O escopo reduzido, o domínio de negócio bem definido e o baixo volume esperado de acessos simultâneos tornam a arquitetura monolítica em camadas mais adequada, reduzindo a complexidade de desenvolvimento, implantação e manutenção sem comprometer os requisitos estabelecidos.

2.2 Banco de dados e transações

Os sistemas gerenciadores de bancos de dados relacionais (SGBDR) organizam as informações em tabelas relacionadas por meio de chaves primárias e estrangeiras. Esse modelo busca reduzir redundâncias e preservar a integridade das informações, garantindo que os relacionamentos entre as entidades permaneçam válidos durante operações de inserção, atualização e exclusão.

As transações constituem um dos principais mecanismos para garantir a confiabilidade dos bancos de dados relacionais. Segundo Elmasri e Navathe (2016), uma transação é um conjunto de operações executadas como uma única unidade lógica de trabalho e deve obedecer às propriedades ACID: Atomicidade, pela qual todas as operações são concluídas ou nenhuma é efetivada; Consistência, que mantém o banco em estado válido antes e após a transação; Isolamento, que impede interferência indevida entre transações concorrentes; e Durabilidade, que preserva os dados confirmados mesmo em caso de falha. Sistemas como o PostgreSQL implementam essas propriedades para assegurar a integridade das informações armazenadas.

No sistema proposto para o gerenciamento de estacionamento, essas características são fundamentais para garantir a integridade das operações realizadas. Quando um veículo deixa o estacionamento, o sistema calcula o valor devido com base no tempo de permanência e registra o pagamento efetuado. Simultaneamente, a vaga deve ser marcada como disponível para um novo veículo. Essas operações precisam ocorrer dentro de uma única transação, de modo que ambas sejam concluídas com sucesso ou ambas sejam canceladas em caso de falha. Sem essa garantia, o sistema poderia registrar o pagamento e manter a vaga indevidamente ocupada, impedindo sua reutilização, ou liberar a vaga sem registrar o pagamento, comprometendo o controle financeiro do estacionamento. Além disso, o cadastro das tarifas deve considerar períodos de vigência, enquanto o valor efetivamente cobrado permanece registrado na própria movimentação do veículo. Dessa forma, alterações futuras nos preços não modificam os valores

das operações já concluídas, permitindo a reconstituição do faturamento e a rastreabilidade das informações armazenadas.

2.3 Usabilidade, acessibilidade e proteção de dados

A usabilidade está ligada à facilidade com que os usuários realizam suas tarefas de forma eficiente e com o menor número possível de erros. Segundo Nielsen (1994), interfaces devem seguir heurísticas que contribuam para uma melhor experiência do usuário. No sistema proposto, destacam-se a visibilidade do status do sistema, permitindo que o operador visualize em tempo real a ocupação das vagas, e o reconhecimento em vez de memorização, uma vez que as informações sobre vagas e veículos permanecem disponíveis na interface.

Além da usabilidade, a acessibilidade deve ser considerada desde o desenvolvimento do sistema, garantindo que pessoas com diferentes necessidades possam utilizar suas funcionalidades de forma autônoma. As Diretrizes de Acessibilidade para Conteúdo Web (WCAG) 2.2, publicadas pelo World Wide Web Consortium (W3C), estabelecem recomendações para que interfaces digitais sejam perceptíveis, operáveis, compreensíveis e robustas, favorecendo o acesso por pessoas com deficiência e promovendo uma experiência mais inclusiva.

Esses princípios orientam a interface utilizada pelos operadores, na qual a organização das informações, as mensagens claras e a navegação simplificada contribuem para reduzir erros durante a operação. Em conformidade com o critério 1.4.1 (Uso de Cor) das WCAG 2.2, o sistema não utiliza apenas cores para indicar o estado das vagas: a interface apresenta também identificação textual e símbolos visuais, permitindo que usuários com deficiência na percepção de cores compreendam corretamente as informações exibidas.

Como o sistema realiza o tratamento de dados pessoais, como a placa do veículo, os registros de entrada e saída e o número de telefone utilizado para o envio de notificações, seu desenvolvimento deve observar os princípios estabelecidos pela Lei Geral de Proteção de Dados Pessoais (Lei nº 13.709/2018). A placa do veículo é considerada dado pessoal por estar relacionada a uma pessoa natural identificada ou identificável, enquanto o histórico de horários de entrada e saída pode revelar padrões de utilização e deslocamento dos condutores. O número de telefone constitui um dado de contato diretamente associado ao condutor e será tratado especificamente para a finalidade de envio das notificações previstas no sistema, mediante a base legal definida para essa finalidade, sendo seu fornecimento opcional. Dessa forma, os dados devem ser coletados para finalidades determinadas, utilizados de acordo com essas finalidades, armazenados de forma segura e mantidos somente pelo período necessário ao

cumprimento das atividades previstas e das obrigações legais. O sistema também mantém registros das operações realizadas, contribuindo para a rastreabilidade e a segurança das informações.

3 PROPOSTA

O presente trabalho propõe o desenvolvimento de um sistema web para o gerenciamento de estacionamento, com o objetivo de informatizar o controle de vagas, registrar a entrada e a saída de veículos, calcular automaticamente o valor da permanência, registrar os pagamentos e fornecer recursos para o controle financeiro da operação. O sistema também contará com uma funcionalidade de notificação automática, destinada ao envio de mensagens pelo WhatsApp quando uma movimentação permanecer aberta além do período configurado. A aplicação será desenvolvida utilizando React na interface web, Node.js com Express no backend e PostgreSQL como sistema gerenciador de banco de dados, adotando a arquitetura cliente-servidor em camadas descrita na seção anterior.

Os requisitos do sistema foram definidos por meio da análise do domínio do problema, da pesquisa bibliográfica e da observação de funcionalidades presentes em sistemas de gerenciamento de estacionamento disponíveis no mercado. Não foram realizadas entrevistas com operadores ou usuários finais, constituindo essa uma limitação do presente trabalho.

O núcleo funcional do sistema constitui a entrega prioritária e compreende o cadastro e gerenciamento de vagas, o registro da entrada e da saída de veículos, o cálculo automático do valor da permanência conforme a tarifa vigente, o pagamento simulado, a atualização da disponibilidade das vagas, o fechamento de caixa e a geração de relatórios financeiros. A notificação automática é uma funcionalidade prevista no escopo, porém condicionada à viabilidade da integração com o serviço de mensagens dentro do período disponível, de modo que eventuais limitações técnicas não comprometam a conclusão do núcleo. Para essa funcionalidade será utilizado um cliente de WhatsApp não oficial, com a lógica de comunicação isolada em um componente específico do backend, o que permite substituir o canal de envio sem alterar as demais regras de negócio.

O número de telefone será informado de forma opcional pelo condutor, sem que sua ausência impeça o registro da entrada, da permanência, da saída ou do pagamento.

Permanecem fora do escopo de implementação as integrações com equipamentos físicos, como cancelas automáticas e reconhecimento automático de placas, além do processamento de pagamentos por instituições financeiras e da emissão de documentos fiscais.

Do pagamento simulado serão armazenados apenas valor, forma e horário, sem número de cartão, CPF ou dados do titular.

3.1 Metodologia

A metodologia adotada caracteriza-se como pesquisa aplicada, de abordagem qualitativa e natureza descritiva, tendo como finalidade o desenvolvimento de uma solução computacional para um problema prático. A fundamentação teórica será construída por meio de pesquisa bibliográfica em livros, documentação oficial, normas e legislação pertinente, subsidiando as decisões adotadas no projeto.

A implementação será realizada de forma incremental, iniciando pela modelagem do banco de dados e pelo desenvolvimento da API REST, seguida da implementação da camada de serviços, da interface web desenvolvida em React e da integração entre os componentes. O módulo financeiro será desenvolvido de forma integrada às movimentações do estacionamento, contemplando o registro dos pagamentos, o fechamento de caixa e os relatórios financeiros. A funcionalidade de notificação automática será implementada no backend por meio de um agendador com intervalo configurável, responsável por verificar as movimentações abertas e acionar o envio das mensagens quando o limite estabelecido for ultrapassado.

A validação será realizada por meio de testes unitários das regras de negócio, incluindo o cálculo da tarifa e as operações financeiras, testes funcionais das principais funcionalidades e testes de integração entre a interface, a API, o banco de dados e o serviço de mensagens. Também será verificado o comportamento da notificação automática, incluindo o respeito ao intervalo configurado e a prevenção de envios duplicados. Por fim, será avaliado o tempo necessário para registrar a saída de um veículo utilizando o sistema, comparando-o com o processo manual, a fim de verificar o potencial ganho de agilidade proporcionado pela solução proposta.

4 CRONOGRAMA DO PROJETO

O Quadro 1 apresenta o cronograma previsto para o desenvolvimento do projeto.

Quadro 1 – Cronograma do projeto.

Tarefa	Ago				Set				Out				Nov				Dez			
	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4
Elaborar a Proposta	X	X																		
Levantar requisitos e modelar o banco			X																	
Testar integração com o WhatsApp				X																
Desenvolver a API REST					X															
Implementar regras de negócio						X														
Desenvolver interface em React							X													
Implementar módulo financeiro								X												
Implementar notificação automática									X											
Integrar e testar o protótipo										X										

Escrever o artigo												X							
Revisar e formatar													X						

Fonte: O Autor

REFERÊNCIAS

BRASIL. Lei nº 13.709, de 14 de agosto de 2018. **Lei Geral de Proteção de Dados Pessoais (LGPD)**. Diário Oficial da União: Brasília, DF, 15 ago. 2018.

ELMASRI, Ramez; NAVATHE, Shamkant B. **Fundamentals of Database Systems**. 7. ed. Boston: Pearson, 2016.

FIELDING, Roy Thomas. **Architectural styles and the design of network-based software architectures**. 2000. Tese (Doutorado em Ciência da Computação) – University of California, Irvine, Irvine, 2000. Disponível em: <https://ics.uci.edu/~fielding/pubs/dissertation/top.htm>. Acesso em: 5 ago. 2026.

NIELSEN, Jakob. **10 usability heuristics for user interface design**. Nielsen Norman Group, 1994. Atualizado em 2020. Disponível em: <https://www.nngroup.com/articles/ten-usability-heuristics/>. Acesso em: 6 ago. 2026.

RIO GRANDE DO SUL. Departamento Estadual de Trânsito. **Frota em circulação no RS**. Porto Alegre: DETRAN/RS, 2024. Dados até fevereiro de 2024. Disponível em: <https://www.detran.rs.gov.br/upload/arquivos/202403/27092408-01-frota-do-rs.pdf>. Acesso em: 4 ago. 2026.

WORLD WIDE WEB CONSORTIUM. **Web Content Accessibility Guidelines (WCAG) 2.2**. [S. l.]: W3C, 2023. Disponível em: <https://www.w3.org/TR/WCAG22/>. Acesso em: 5 ago. 2026.